

University
Politehnica
of Bucharest



Faculty of
Automatic
Control and
Computers



Computer
Science and
Engineering
Department

Column-Oriented Databases
Cassandra

Ciprian-Octavian Truică
Florin Rădulescu



Overview

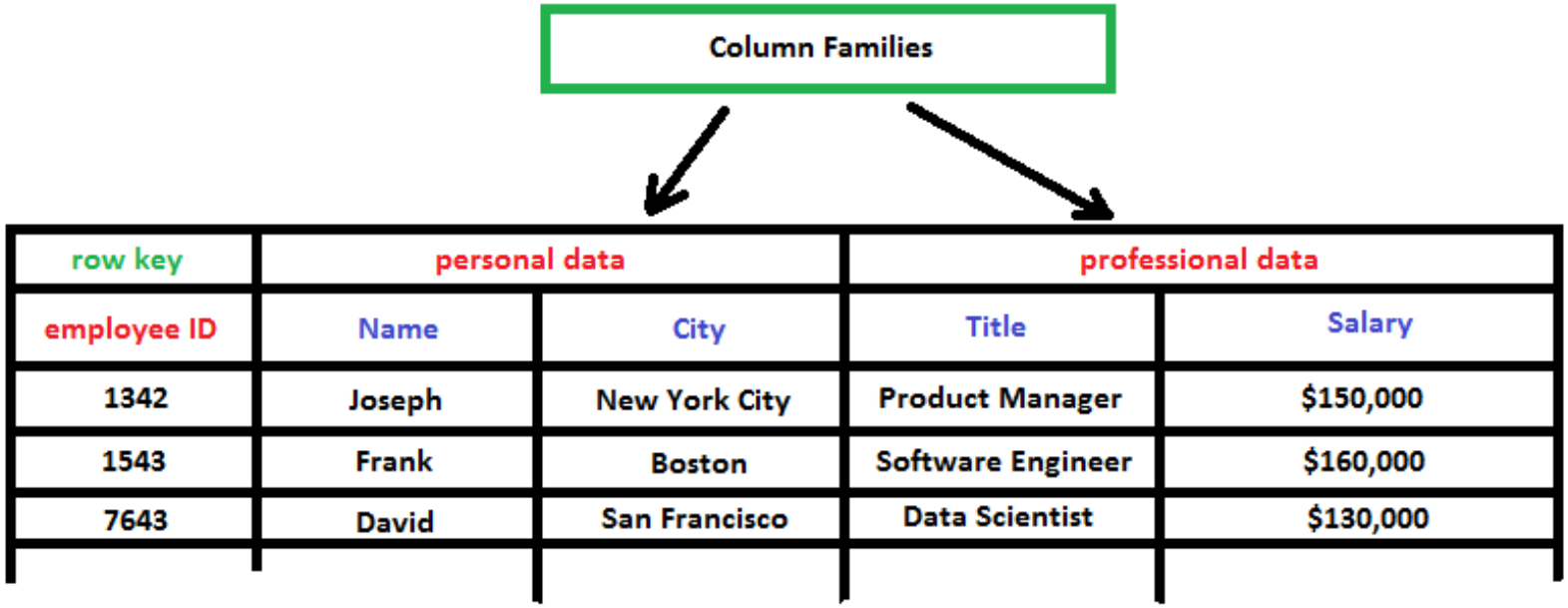
- Column-Oriented Databases
- Cassandra



Column Stores

- Column-Oriented Databases Management Systems (CODBMS) stores data using tables, rows, and columns.
- Unlike a relational database, the names and format of the columns can vary from row to row in the same table.
- Columns are grouped in Column Families.
- CODBMSs are also called Wide Column stores or Columnar Database.

Column Stores



row key	personal data		professional data	
employee ID	Name	City	Title	Salary
1342	Joseph	New York City	Product Manager	$150,000
1543	Frank	Boston	Software Engineer	$160,000
7643	David	San Francisco	Data Scientist	$130,000

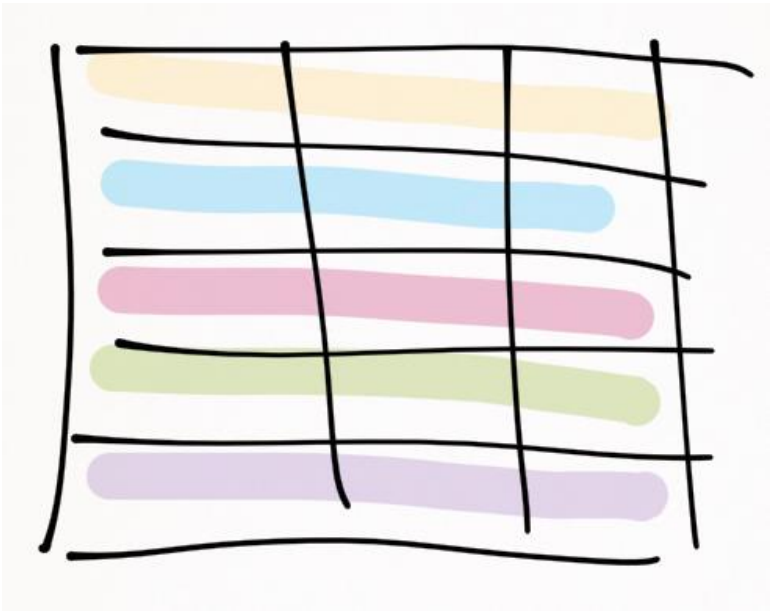


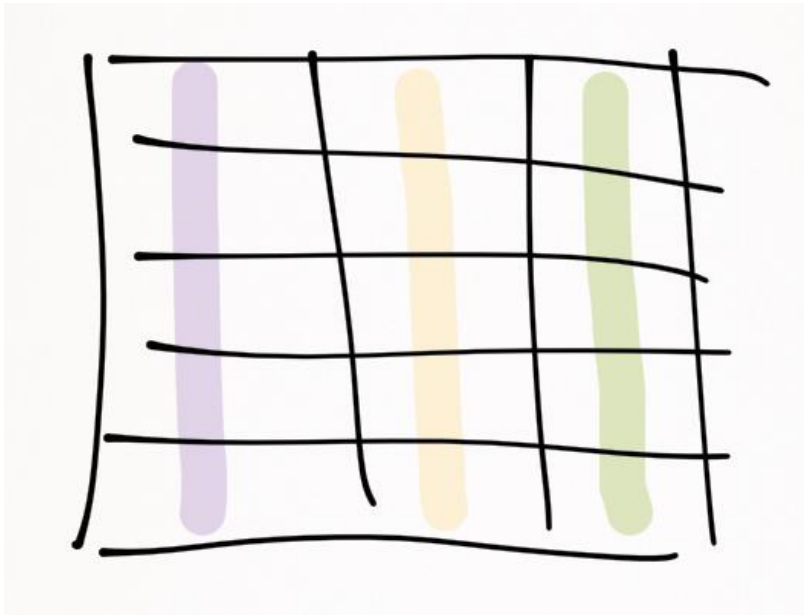
Column Stores

- CODBMS Databases uses one of two approaches to partitioning data on disk:
  - Row-Oriented: partition tables by rows
  - Column-Oriented: partition tables by columns/column families

Column-Oriented Databases

- Partitioning: Row-Oriented vs. Column-Oriented







Column Stores

- CODBMS disk storage approach: Row-Oriented partitioning
  - Predictability: tuples address can be predicted without looking at the data.
  - Locality: reduce the probability to read the data from memory cache or the same block on disk.
  - Read operations: to read one attribute the entire tuple must be read.

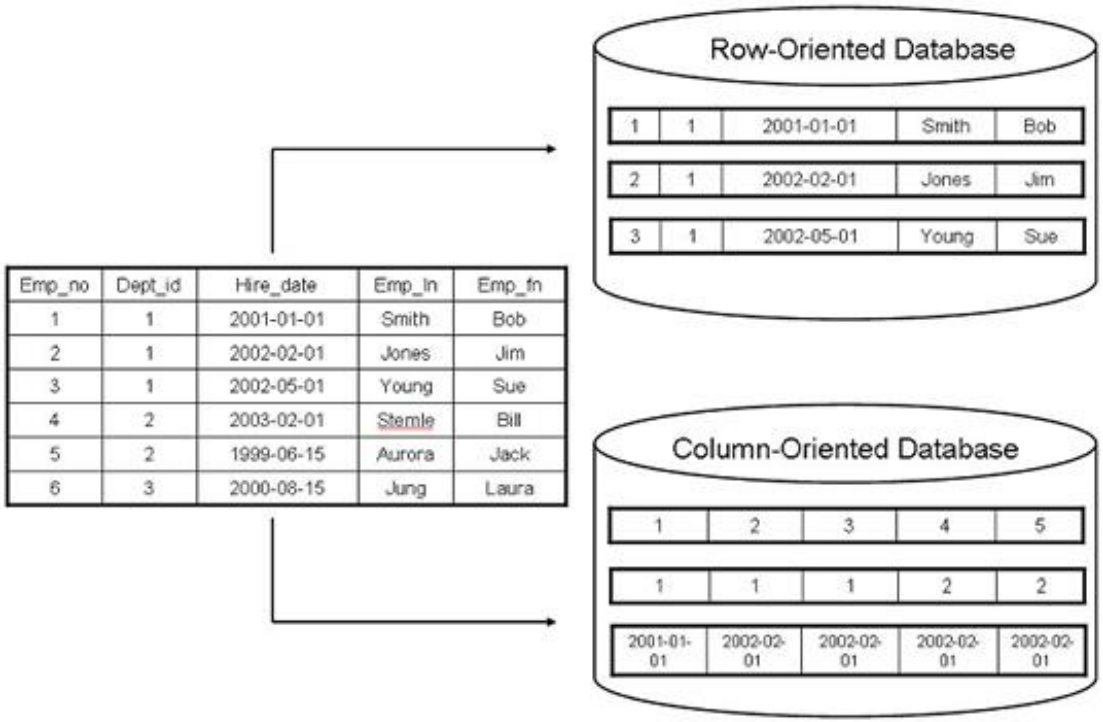


Column Stores

- CODBMS disk storage approach: Column-Oriented partitioning
  - Predictability: if the data uses fixed length types than we can predict exactly where we find the data, but in the case of tuples with variable length, we cannot.
  - Locality: the probability to hit the cache (or block) is very high during iteration.
  - Read operations: read only the data needed.

Column Stores

- Partitioning: Row-Oriented vs. Column-Oriented



Emp_no	Dept_id	Hire_date	Emp_ln	Emp_fn
1	1	2001-01-01	Smith	Bob
2	1	2002-02-01	Jones	Jim
3	1	2002-05-01	Young	Sue
4	2	2003-02-01	Stemle	Bill
5	2	1999-06-15	Aurora	Jack
6	3	2000-08-15	Jung	Laura



Cassandra

- Cassandra
  - First developed by Facebook to solve the inbox search problem.
  - The team wanted to address the following problems:
    - Handle huge volumes of data in the form of message copies
    - Reverse indices of messages
    - Many random reads
    - Many simultaneous random writes.



Cassandra

- Cassandra
  - Is open source
  - Is Distributed and Decentralized
  - Is Elastically Scalable
  - Is Highly Available and Fault-Tolerant
  - Uses Tunable Consistency
  - High Performance
  - Row-Oriented Partitioning



Cassandra

- Cassandra is distributed
  - Cassandra is capable of running on multiple machines while appearing to users as a unified whole.
  - Much of its design and code base is specifically engineered toward not only making it work across many different machines, but also for optimizing performance across multiple data center racks, and even for a single Cassandra cluster running across geographically dispersed data centers.



Cassandra

- Cassandra is decentralized
  - There is no single point of failure.
  - All of the nodes in a Cassandra cluster function exactly the same. This is sometimes referred to as “server symmetry”.
  - Because they are all doing the same thing, by definition there can’t be a special host that is coordinating activities, as with the master/slave setup



Cassandra

- Cassandra is Elastic Scalable
  - Elastic scalability refers to a special property of horizontal scalability.
  - It means that your cluster can seamlessly scale up and scale back down.
  - The cluster must be able to accept new nodes that can begin participating by getting a copy of some or all of the data and start serving new user requests without major disruption or reconfiguration of the entire cluster.



Cassandra

- Cassandra is Elastic Scalable
  - There is no need to restart any process.
  - There is no need to change any application queries.
  - There is no need to manually rebalance the data.
  - Cassandra will find, start and send work to any new machine added to the cluster.



Cassandra

- Cassandra is Highly Available and Fault Tolerance
  - Failed nodes can be replaced in the cluster with no downtime.
  - Data can be replicated to multiple data centers to offer improved local performance.
  - Replication helps to prevent downtime if one data center experiences a catastrophe.



Cassandra

- Cassandra uses Tunable Consistency
  - Cassandra allows the clients to choose the level of consistency
  - The consistency level is a setting that clients must specify on every operation and that allows you to decide how many replicas in the cluster must acknowledge a write operation or respond to a read operation in order to be considered successful.
  - Cassandra's consistency uses the replication factor.
  - The replication factor is a trade-off between how much you want to pay in performance to gain more consistency.
  - The replication factor value is equal to the number of nodes in the cluster where a modification (insert, update, delete) will propagate to.



Cassandra

- Cassandra uses Tunable Consistency
  - The consistency level can be equal to the replication factor.
  - Thus, the gain is stronger consistency at the cost of synchronous blocking operations that wait for all nodes to be updated and declare success before returning.



Cassandra

- Cassandra is a High Performance database
  - Cassandra was designed specifically from the ground up to take full advantage of multiprocessor/multi-core machines.
  - It can run across many dozens of these machines housed in multiple data centers.
  - It scales consistently and seamlessly to hundreds of terabytes.
  - Cassandra has been shown to perform exceptionally well under heavy load.



Cassandra

- Cassandra is a High Performance database
  - It consistently can show very fast throughput for writes per second on basic commodity computers, whether physical hardware or virtual machines.
  - As more servers are added, all of Cassandra's desirable properties can be maintained without sacrificing performance.



Cassandra

- Cassandra's Row-Oriented partitioning
  - Cassandra's data model can be described as a partitioned row store.
  - Data is stored in sparse multidimensional hashtables.
  - "Sparse" means that for any given row you can have one or more columns, but each row doesn't need to have all the same columns as other rows like it (as in a relational model).
  - "Partitioned" means that each row has a unique key which makes its data accessible, and the keys are used to distribute the rows across multiple data stores.



Cassandra

- Cassandra is a good fit for
  - Large Deployments
  - Lots of Writes, Statistics, and Analytics
  - Geographical Distribution
  - Evolving Applications



Cassandra

- Large Deployments:
  - Cassandra is a good fit if the application is expected to require dozens of nodes.
  - Cassandra is a good fit for applications that needs high availability
  - Cassandra is a good fit for applications that are geographical distributed



Cassandra

- Lots of Writes, Statistics, and Analytics
  - Before choosing a database, consider your application from the perspective of the ratio of reads to writes.
  - Cassandra is optimized for excellent throughput on writes.
  - One of Cassandra's primary feature is the ability to handle application workloads that require high performance at significant write volumes with many concurrent client threads



Cassandra

- Lots of Writes, Statistics, and Analytics
  - Cassandra is a good choice for storing user activity updates, social network usage, recommendations/reviews, and application statistics.
  - Cassandra is also a good choice when the application involve lots of writing with less predictable read operations.
  - When updates can occur unevenly with sudden spikes.



Cassandra

- Lots of Writes, Statistics, and Analytics
  - Cassandra has been used to create a variety of applications, e.g.
    - Windowed time-series store,
    - An inverted index for document searching,
    - A distributed job priority queue, etc.



Cassandra

- Geographical Distribution
  - Cassandra has out-of-the-box support for geographical distribution of data.
  - Cassandra can be easily configure to replicate data across multiple data centers.
  - Cassandra could be a great fit if you have a globally deployed application that could see a performance benefit from putting the data near the user.



Cassandra

- Evolving Applications
  - Cassandra might be a good fit if the application is evolving rapidly given its support for flexible schemas.
  - Cassandra makes it easy to keep your database in step with application changes as you rapidly deploy.



Cassandra – Data Model

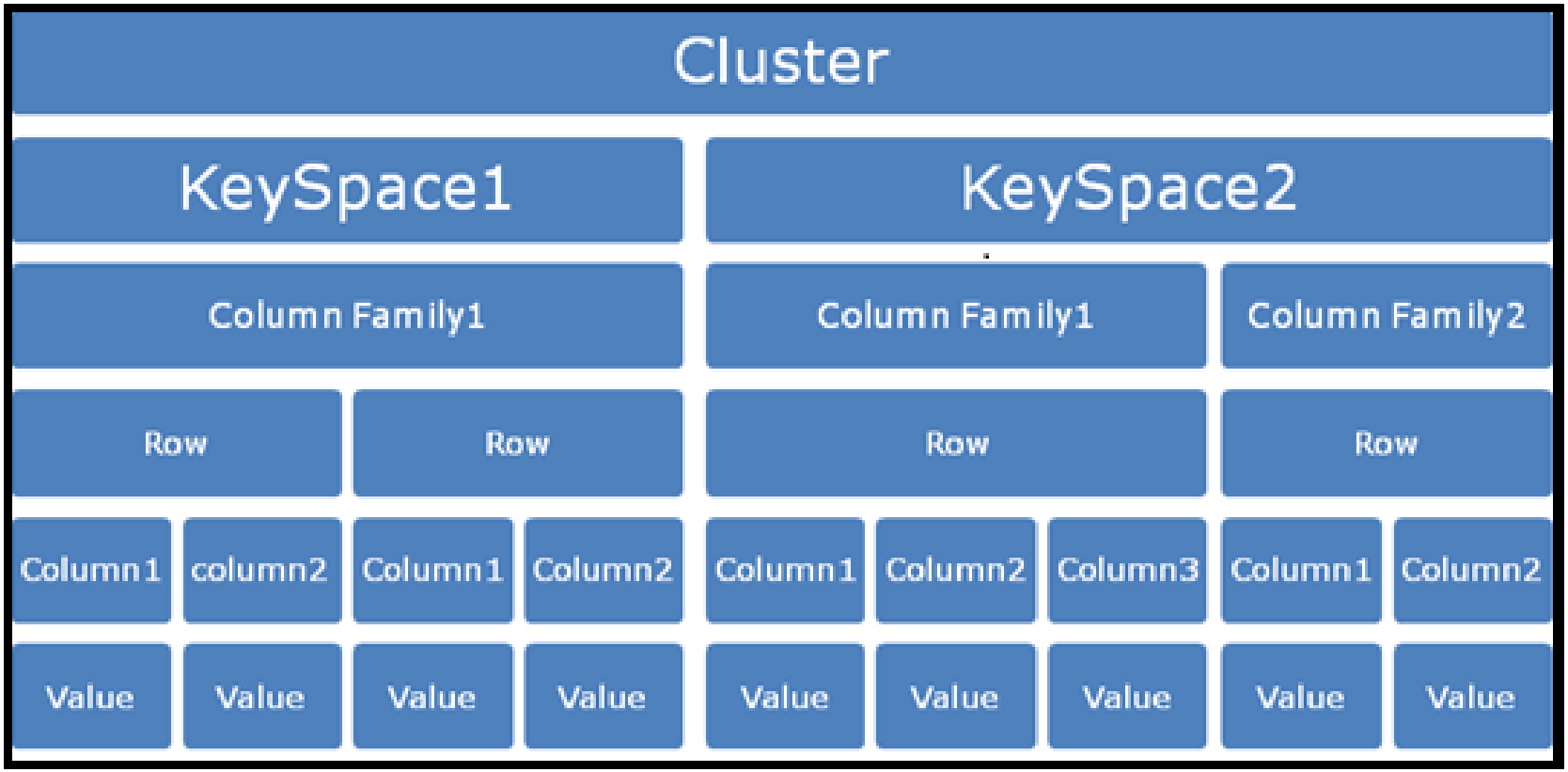
- Cassandra data model contains the following structures:
  - Cluster
  - Keyspace
  - Table



Cassandra – Data Model

- An instance of Cassandra typically consists of only one table which represents a distributed multi-dimensional map indexed by a key.
- A table is structured by the following dimensions:
  - Rows
  - Row Key (Partition Key)
  - Column-families
  - Columns
  - Super-columns

Cassandra – Data Model





Cassandra – Data Model

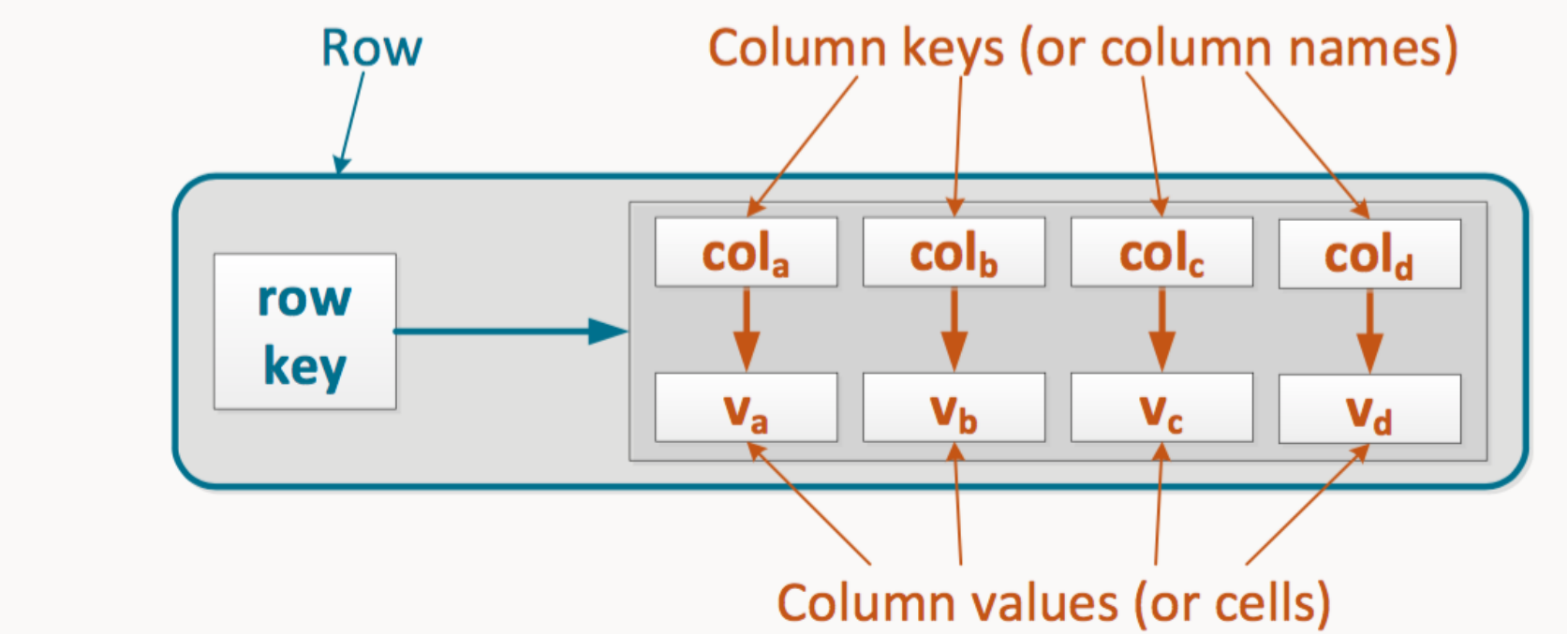
- Rows
  - A row is the smallest unit that stores related data in Cassandra.
  - Operations on rows are atomic per replica no matter how many columns are being read or written.
  - Rows are identified by a string-key of arbitrary length.
  - A row key uniquely identifies a row in a column family.
  - A row stores pairs of column keys and column values.
  - A column key uniquely identifies a column value in a row.
  - A column value stores one value or a collection of values



Cassandra – Data Model

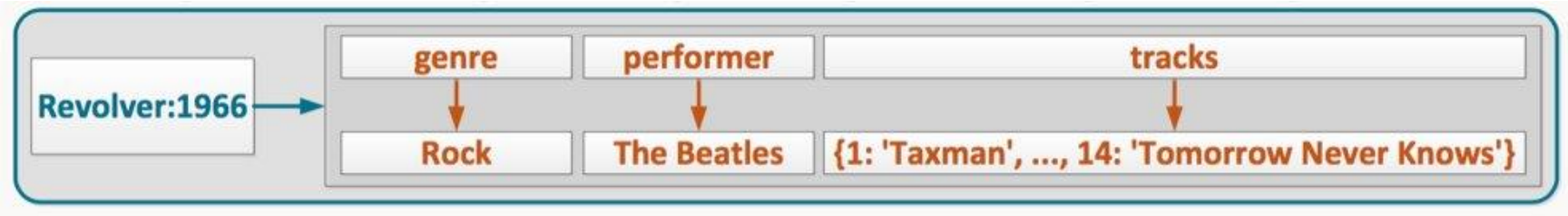
- Rows may be described as skinny or wide
  - Skinny row: has a fixed, relatively small number of column keys.
  - Wide row: has a relatively large number of column keys (hundreds or thousands); this number may increase as new data values are inserted

Cassandra – Data Model



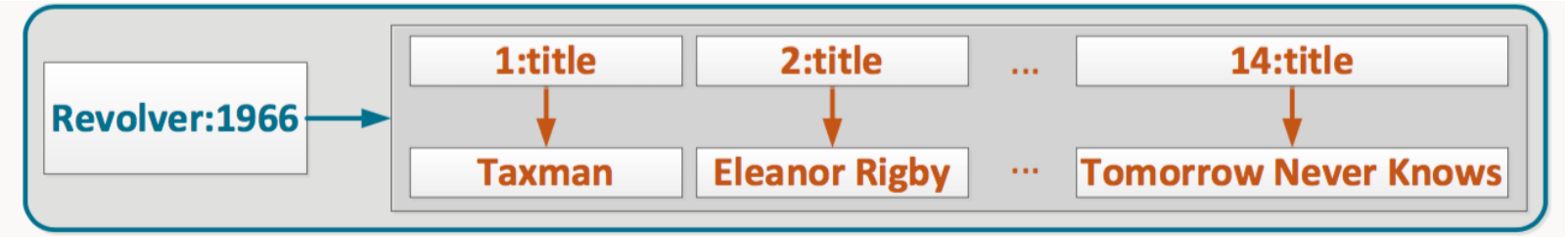
Cassandra – Data Model

- Key
  - Composite row key: multiple components separated by colon.



Cassandra – Data Model

- Key
  - Composite column key: multiple components separated by colon.



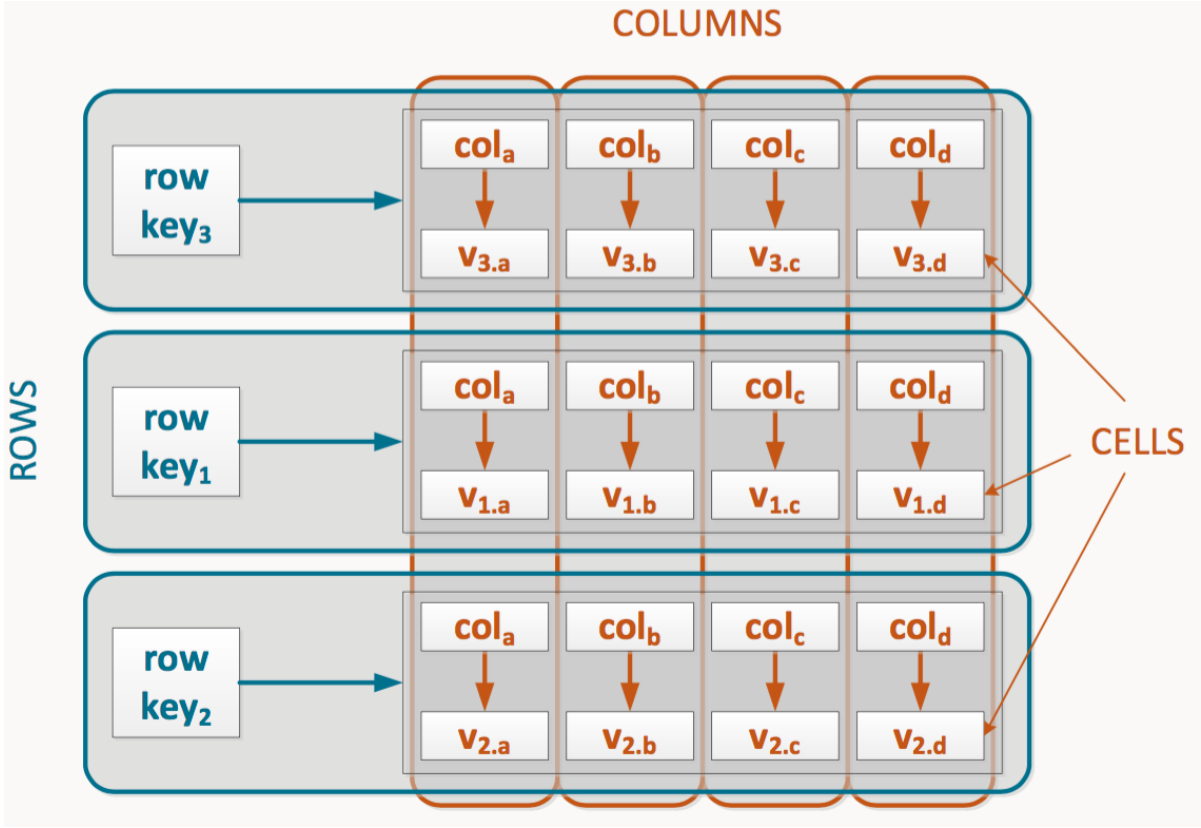


Cassandra – Data Model

- Column families
  - Set of rows with a similar structure
  - Can occur in an arbitrary number per row.
  - Have to be defined in advance, i.e. before a cluster of servers comprising a Cassandra instance is launched.
  - The number of column families per table is not limited; however, it is expected that only a few of them are specified.
  - Consists of columns and super columns which can be added dynamically (i.e. at runtime) to column families and are not restricted in number.

Cassandra – Data Model

- Column-families





Cassandra – Data Model

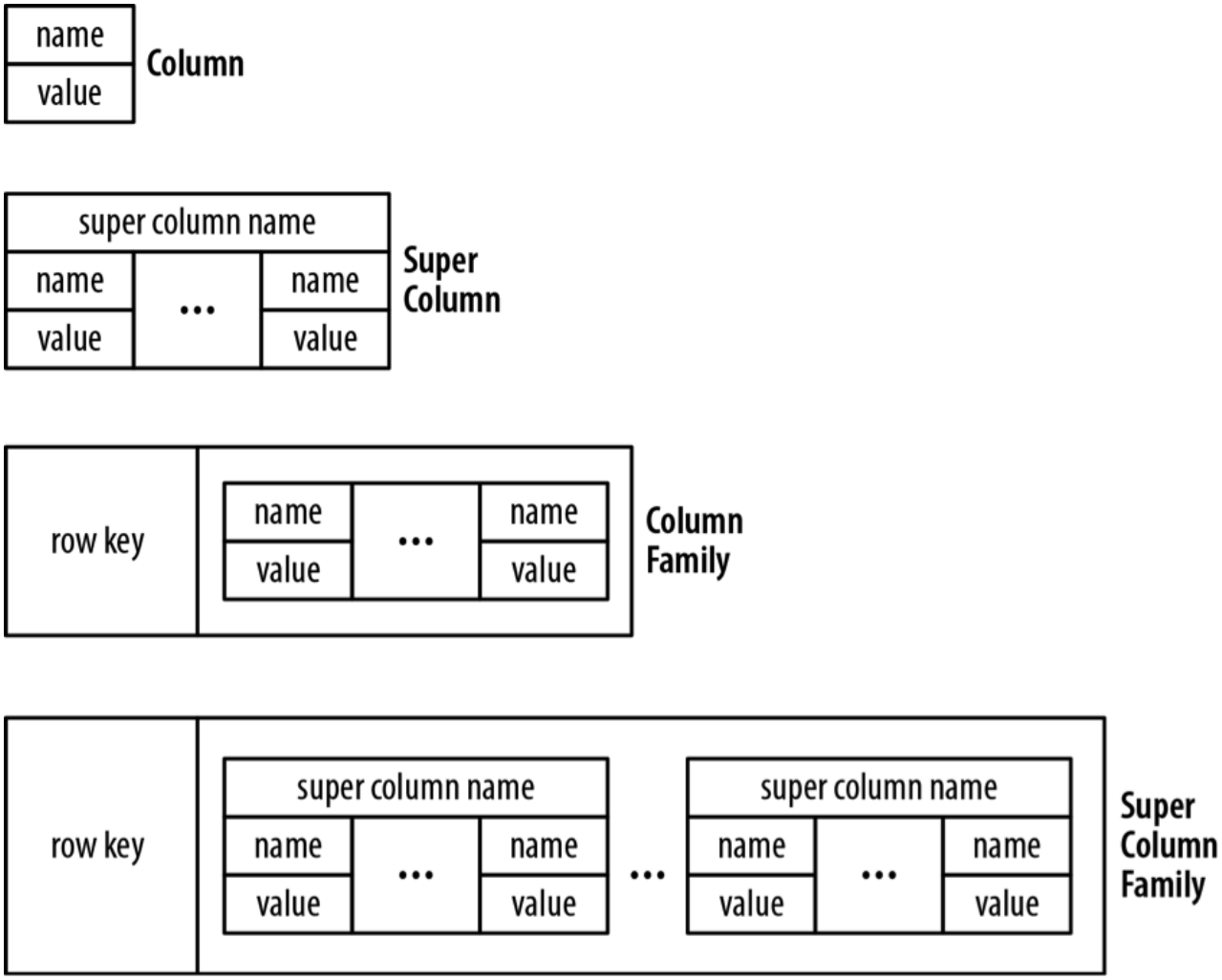
- Columns
  - Must have a name.
  - Store a number of values per row which are identified by a timestamp.
  - Each row in a table can have a different number of columns, so a table cannot be thought of as a rectangle.
  - Client applications may specify the ordering of columns within a column family and super column which can either be by name or by timestamp.



Cassandra – Data Model

- Super columns
  - Must have a name.
  - Can have an arbitrary number of columns associated with them.
  - The number of columns per super column may differ per row.
- Super column families
  - When rows contain column families, then the collection is called a super column family.

Cassandra – Data Model





Cassandra – Data Model

- Clusters
  - Cassandra database is specifically designed to be distributed over several machines operating together that appear as a single instance to the end user.
  - The outermost structure in Cassandra is the cluster, sometimes called the ring, because Cassandra assigns data to nodes in the cluster by arranging them in a ring.



Cassandra – Data Model

- Keyspaces
  - A cluster is a container for keyspaces.
  - A keyspace is the outermost container for data in Cassandra, corresponding closely to a relational database.
  - In the same way that a database is a container for tables in the relational model, a keyspace is a container for tables in the Cassandra data model.
  - Like a relational database, a keyspace has a name and a set of attributes that define keyspace-wide behavior.

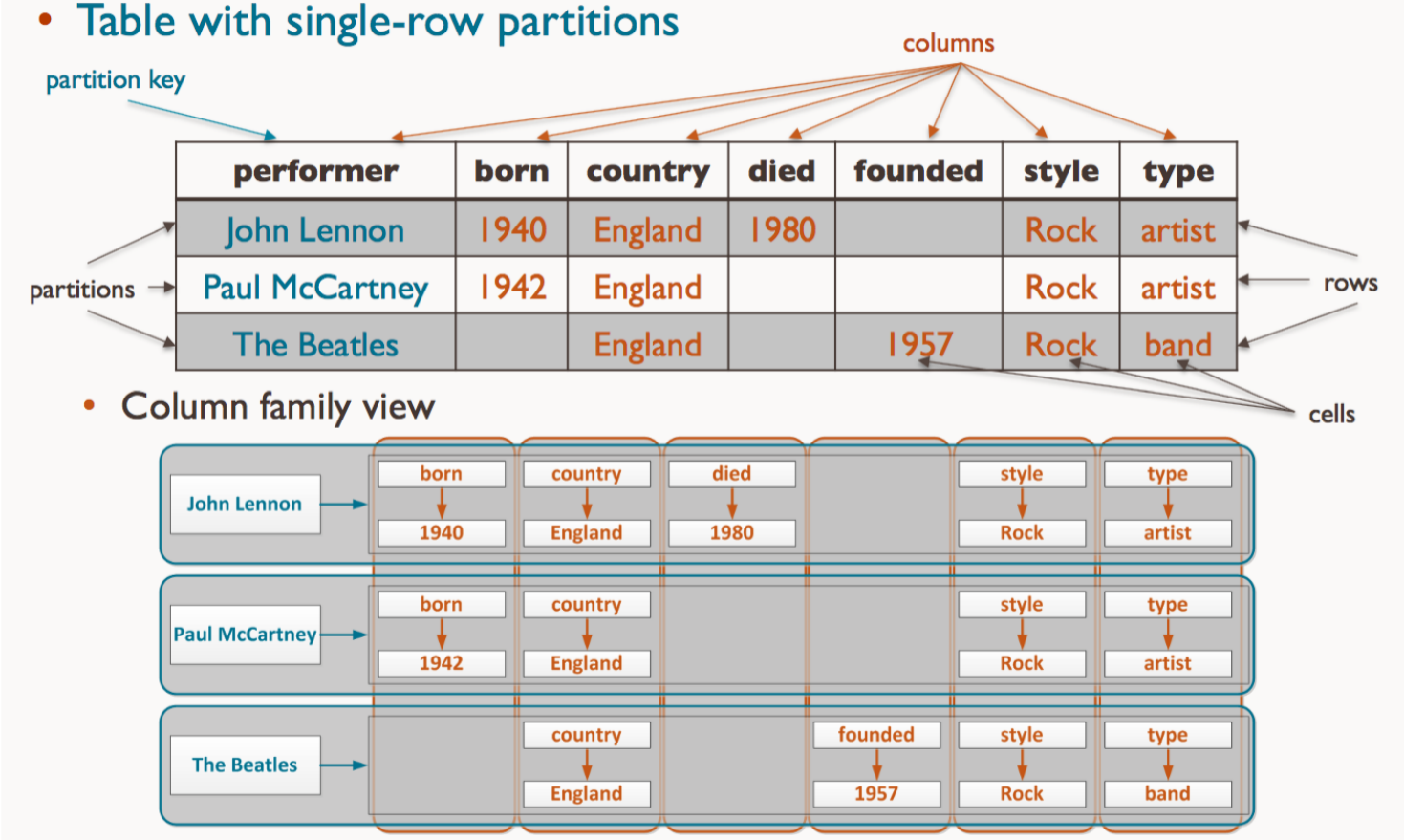


Cassandra – Data Model

- Tables
  - A table is a container for an ordered collection of rows, each of which is itself an ordered collection of columns.
  - The ordering is determined by the columns, which are identified as keys.
  - When you write data to a table in Cassandra, you specify values for one or more columns.
  - That collection of values is called a row.
  - At least one of the values you specify must be a primary key that serves as the unique identifier for that row.

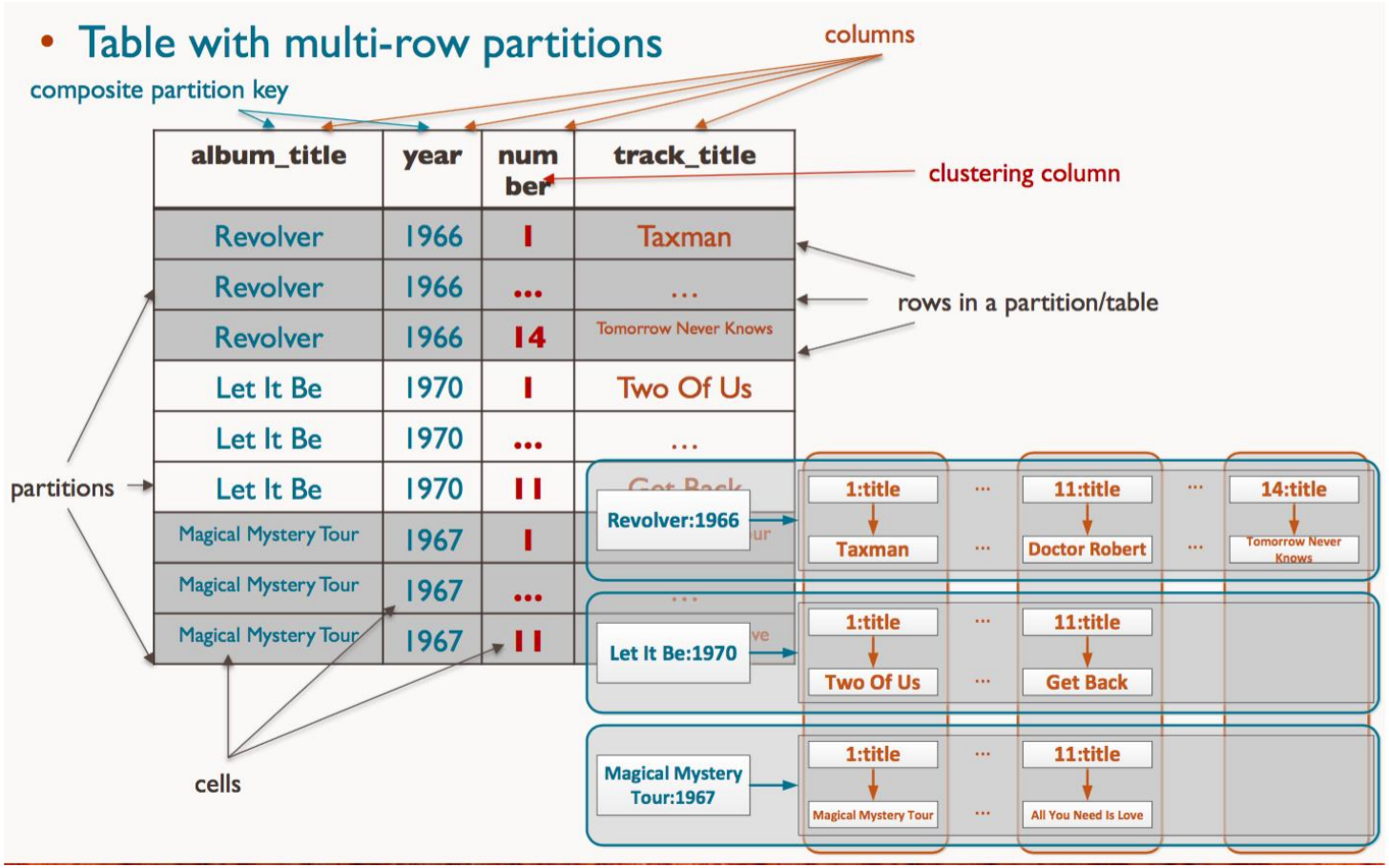
Cassandra – Data Model

- Table with single-row partitions



Cassandra – Data Model

- Table with multi-row partitions





Cassandra – Architecture

- Architecture
  - Data Centers and Racks
  - Gossip and Failure Detection
  - Rings and Tokens
  - Vnodes (virtual nodes)
  - Partitioners
  - Replication Strategies
  - Consistency Levels
  - Queries and Coordinator Nodes
  - Commit Logs, Memtables, and SSTables
  - Caching
  - HInted Handoff

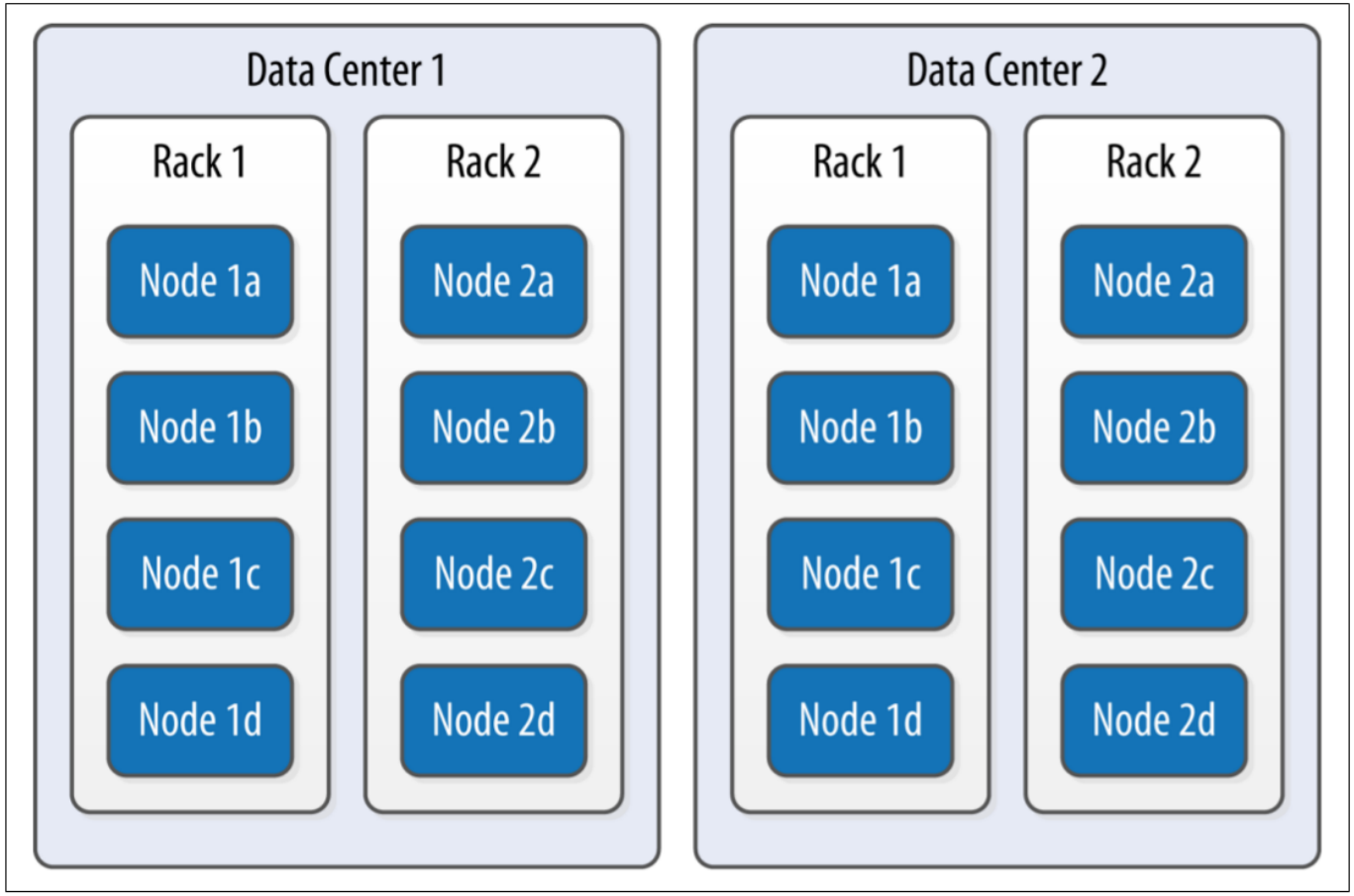


Cassandra – Architecture

- Data Centers and Racks
  - Cassandra is frequently used in systems spanning physically separate locations.
  - Cassandra provides two levels of grouping that are used to describe the topology of a cluster: data center and rack.
  - A rack is a logical set of nodes in close proximity to each other, perhaps on physical machines in a single rack of equipment.
  - A data center is a logical set of racks, perhaps located in the same building and connected by reliable network.

Cassandra – Architecture

- Data Centers and Racks





Cassandra – Architecture

- Data Centers and Racks
  - Cassandra leverages the information you provide about your cluster's topology to determine where to store data, and how to route queries efficiently.
  - Cassandra tries to store copies of your data in multiple data centers to maximize availability and partition tolerance, while preferring to route queries to nodes in the local data center to maximize performance.



Cassandra – Architecture

- Gossip and Failure Detection
  - Cassandra uses a gossip protocol to support decentralization and partition tolerance.
  - The gossip protocol allows each node to keep track of state information about the other nodes in the cluster.
  - The gossiper runs every second on a timer.
  - The gossip protocols take their name from the concept of human gossip, a form of communication in which peers can choose with whom they want to exchange information.



Cassandra – Architecture

- The gossip protocols
  - Generally assume a faulty network
  - Are commonly employed in very large, decentralized network systems
  - Are often used as an automatic mechanism for replication in distributed databases.
  - When a server node is started, it registers itself with the gossiper to receive endpoint state information.
  - Because Cassandra gossip is used for failure detection, the Gossiper maintains a list of nodes that are alive and dead.



Cassandra – Architecture

- The gossiper works as follows:

1. Once per second, the gossiper will choose a random node in the cluster and initialize a gossip session with it. Each round of gossip requires three messages.
2. The gossip initiator sends its chosen node a message.
3. When the node receives this message, it returns a ack message to the initiator.
4. When the initiator receives the ack message from the node, it sends the node an ack message to complete the round of gossip.



Cassandra – Architecture

- Gossip and Failure Detection
  - When the gossiper determines that another endpoint is dead, it "convicts" that end-point by marking it as dead in its local list and logging that fact.
  - Cassandra has robust support for failure detection, as specified by a popular algorithm for distributed computing called Phi Accrual Failure Detection.



Cassandra – Architecture

- Phi Accrual Failure Detectors
  - Accrual Failure Detectors output a value associated with each process (or node).
  - This value is called Phi.
  - The value is output in a manner that is designed from the ground up to be adaptive in the face of volatile network conditions, so it's not a binary condition that simply checks whether a server is up or down.
  - The Phi convict threshold in the configuration adjusts the sensitivity of the failure detector.



Cassandra – Architecture

- Phi Accrual Failure Detectors
  - Lower values increase the sensitivity and higher values decrease it, but not in a linear fashion.
  - The Phi value refers to a level of suspicion that a server might be down.
  - Applications such as Cassandra that employ an AFD can specify variable conditions for the Phi value they emit.
  - Cassandra can generally detect a failed node in about 10 seconds using this mechanism.



Cassandra – Architecture

- Cassandra's accrual failure
  - Cassandra's accrual failure detection is based on two primary ideas.
  - The first general idea is that failure detection should be flexible, which is achieved by decoupling it from the application being monitored.
  - The second and more novel idea challenges the notion of traditional failure detectors, which are implemented by simple "heartbeats" and decide whether a node is dead or not dead based on whether a heartbeat is received or not.
  - But accrual failure detection decides that this approach is naive, and finds a place in between the extremes of dead and alive - a suspicion level.



Cassandra – Architecture

- Cassandra's accrual failure
  - Therefore, the failure monitoring system outputs a continuous level of "suspicion" regarding how confident it is that a node has failed.
  - This is desirable because it can take into account fluctuations in the network environment.
  - For example, just because one connection gets caught up doesn't necessarily mean that the whole node is dead.
  - So suspicion offers a more fluid and proactive indication of the weaker or stronger possibility of failure based on interpretation (the sampling of heartbeats), as opposed to a simple binary assessment.



Cassandra – Architecture

- Rings and Tokens
  - Cassandra represents the data managed by a cluster as a ring.
  - Each node in the ring is assigned one or more ranges of data described by a token, which determines its position in the ring.
  - A token is a 64-bit integer ID used to identify each partition.
  - This gives a possible range for tokens from –2^63 to 2^63 –1.



Cassandra – Architecture

- Rings and Tokens
  - A node claims ownership of the range of values less than or equal to each token and greater than the token of the previous node.
  - The node with lowest token owns the range less than or equal to its token and the range greater than the highest token, which is also known as the "wrapping range".
  - In this way, the tokens specify a complete ring.
  - This particular arrangement is structured such that consecutive token ranges are spread across nodes in different racks.

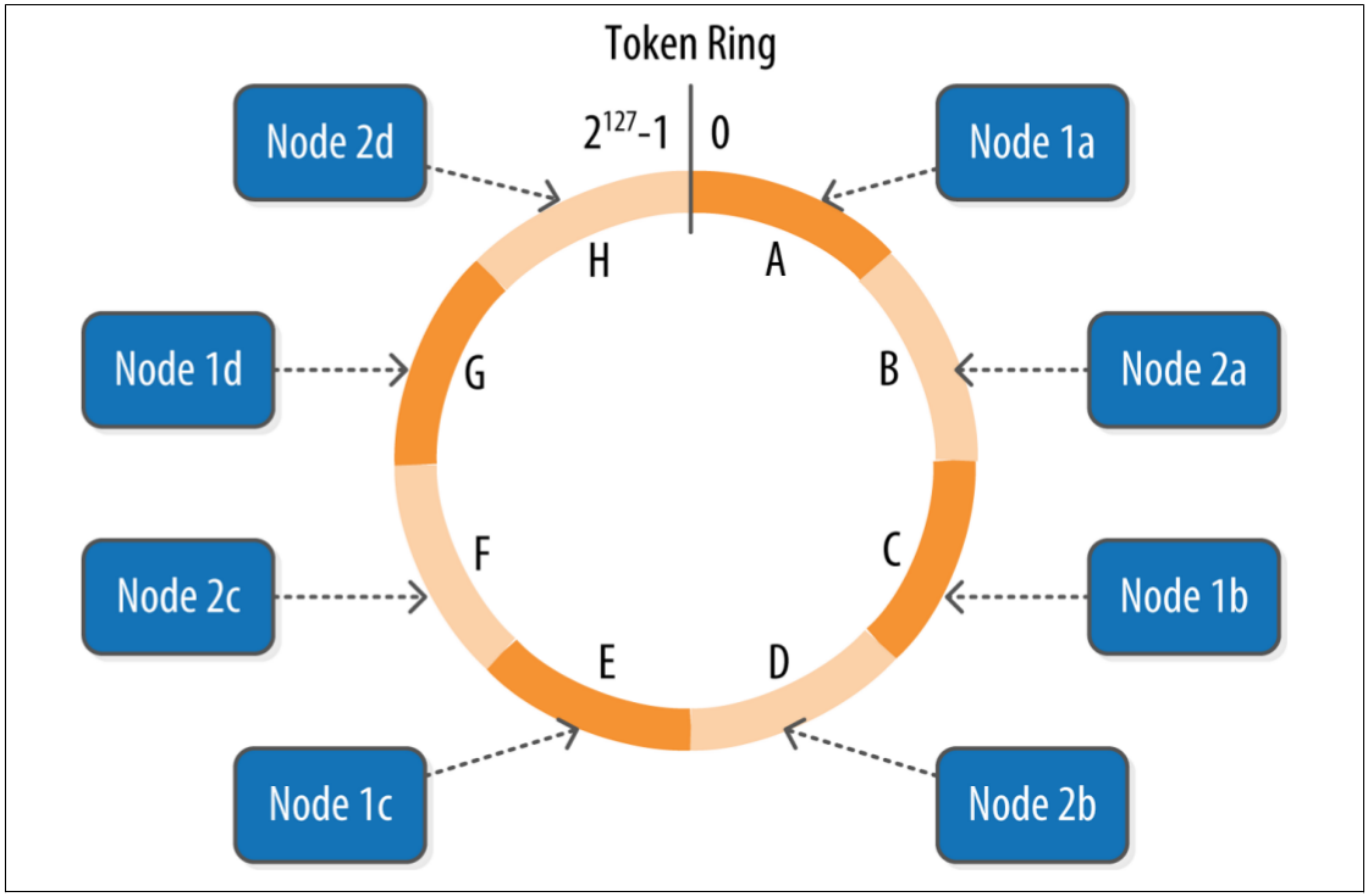


Cassandra – Architecture

- Rings and Tokens
  - Data is assigned to nodes by using a hash function to calculate a token for the partition key.
  - This partition key token is compared to the token values for the various nodes to identify the range, and therefore the node, that owns the data.

Cassandra – Architecture

- Rings and Tokens





Cassandra – Architecture

- Virtual Nodes
  - Cassandra v1.2 release introduced the concept of virtual nodes, also called vnodes for short.
  - Instead of assigning a single token to a node, the token range is broken up into multiple smaller ranges.
  - Each physical node is then assigned multiple tokens.
  - By default, each node will be assigned 256 of these tokens, meaning that it contains 256 virtual nodes.



Cassandra – Architecture

- Virtual Nodes
  - Virtual nodes have been enabled by default since Cassandra v2.0.
  - Vnodes make it easier to maintain a cluster containing heterogeneous machines.
  - For nodes in your cluster that have more computing resources available to them, you can increase the number of vnodes by setting the num_tokens property in the cassandra.yaml file.
  - Conversely, you might set num_tokens lower to decrease the number of vnodes for less capable machines.



Cassandra – Architecture

- Virtual Nodes
  - Cassandra automatically handles the calculation of token ranges for each node in the cluster in proportion to their num_tokens value.
  - A further advantage of virtual nodes is that they speed up some of the more heavy-weight Cassandra operations such as bootstrapping a new node, decommissioning a node, and repairing a node.
  - This is because the load associated with operations on multiple smaller ranges is spread more evenly across the nodes in the cluster.



Cassandra – Architecture

- Partitioners
  - A partitioner determines how data is distributed across the nodes in the cluster.
  - Cassandra stores data in wide rows, or "partitions".
  - Each row has a partition key that is used to identify the partition.
  - A partitioner, then, is a hash function for computing the token of a partition key.
  - Each row of data is distributed within the ring according to the value of the partition key token.



Cassandra – Architecture

- Partitioners
  - Cassandra provides several different partitioners that use a distributed hash table (DHT).
  - A DHT is a class of a decentralized distributed system that provides a lookup service similar to a hash table.
  - For hashing it uses a Java implementation of the murmur algorithm that generates 64-bit hashes.
  - Because of Cassandra's generally pluggable design, you can also create your own partitioner.



Cassandra – Architecture

- Replication Strategies
  - A node serves as a replica for different ranges of data.
  - If one node goes down, other replicas can respond to queries for that range of data.
  - Cassandra replicates data across nodes in a manner transparent to the user.
  - The replication factor is the number of nodes in your cluster that will receive copies (replicas) of the same data.



Cassandra – Architecture

- Replication Strategies
  - If the replication factor is 3, then three nodes in the ring will have copies of each row.
  - The first replica will always be the node that claims the range in which the token falls.
  - The remainder of the replicas are placed according to the replication strategy (or replica placement strategy).



Cassandra – Architecture

- Cassandra provides two primary replication strategies:
  - The SimpleStrategy
    - Places replicas at consecutive nodes around the ring
    - It starts with the node indicated by the partitioner.
  - The NetworkTopologyStrategy
    - That allows the administrator to specify a different replication factor for each data center.
    - Within a data center, it allocates replicas to different racks in order to maximize availability.



Cassandra – Architecture

- Consistency Levels
  - Cassandra provides tuneable consistency levels that allow you to make trade-offs between consistency, availability, and partition tolerance at a fine-grained level.
  - Cassandra lets the client specify a consistency level on each read or write query that indicates how much consistency you require.
  - A higher consistency level means that more nodes need to respond to a read or write query, ensuring that the values present on each replica are the same.



Cassandra – Architecture

- Consistency Levels
  - For read queries, the consistency level specifies how many replica nodes must respond to a read request before returning the data.
  - For write operations, the consistency level specifies how many replica nodes must respond for the write to be reported as successful to the client.
  - Because Cassandra is eventually consistent, updates to other replica nodes may continue in the background.



Cassandra – Architecture

- Consistency Levels
  - The available consistency levels include ONE, TWO, and THREE, each of which specify an absolute number of replica nodes that must respond to a request.
  - The QUORUM consistency level requires a response from a majority of the replica nodes (expressed as "replication factor / 2 + 1").
  - The ALL consistency level requires a response from all of the replicas.
  - For both reads and writes, the consistency levels of
    - ONE, TWO, and THREE are considered weak.
    - QUORUM and ALL are considered strong.



Cassandra – Architecture

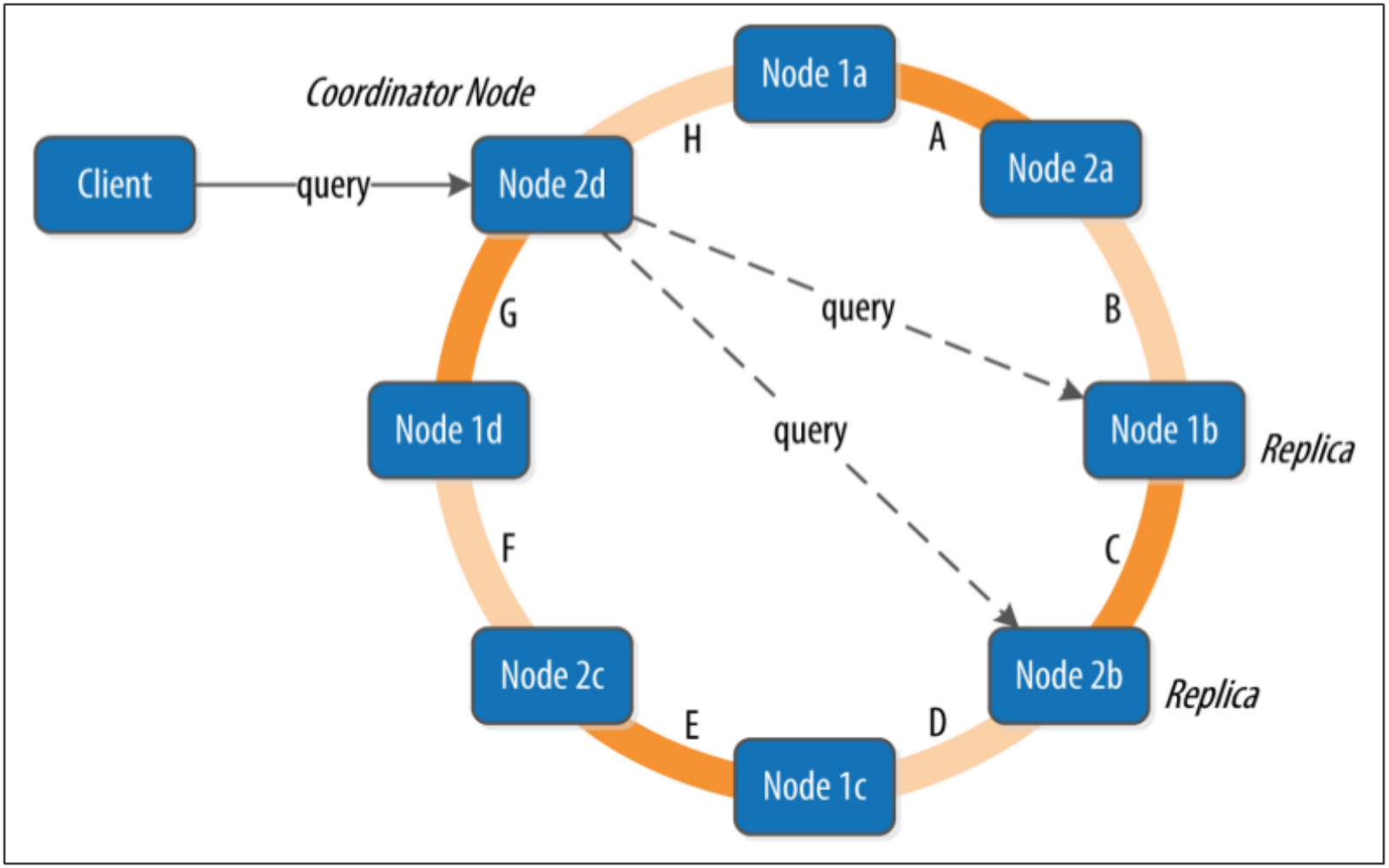
- Consistency Levels
  - Consistency is tuneable in Cassandra because clients can specify the desired consistency level on both reads and writes.
  - There is an equation that is popularly used to represent the way to achieve strong consistency in Cassandra: R + W > N = strong consistency.
  - In this equation, R, W, and N are the read replica count, the write replica count, and the replication factor, respectively.
  - All client reads will see the most recent write in this scenario, and you will have strong consistency.



Cassandra – Architecture

- Queries and Coordinator Nodes
  - A client may connect to any node in the cluster to initiate a read or write query.
  - This node is known as the coordinator node.
  - The coordinator identifies which nodes are replicas for the data that is being written or read and forwards the queries to them.
  - For a write, the coordinator node contacts all replicas, as determined by the consistency level and replication factor, and considers the write successful when a number of replicas commensurate with the consistency level acknowledge the write.
  - For a read, the coordinator contacts enough replicas to ensure the required consistency level is met, and returns the data to the client.

Cassandra – Architecture



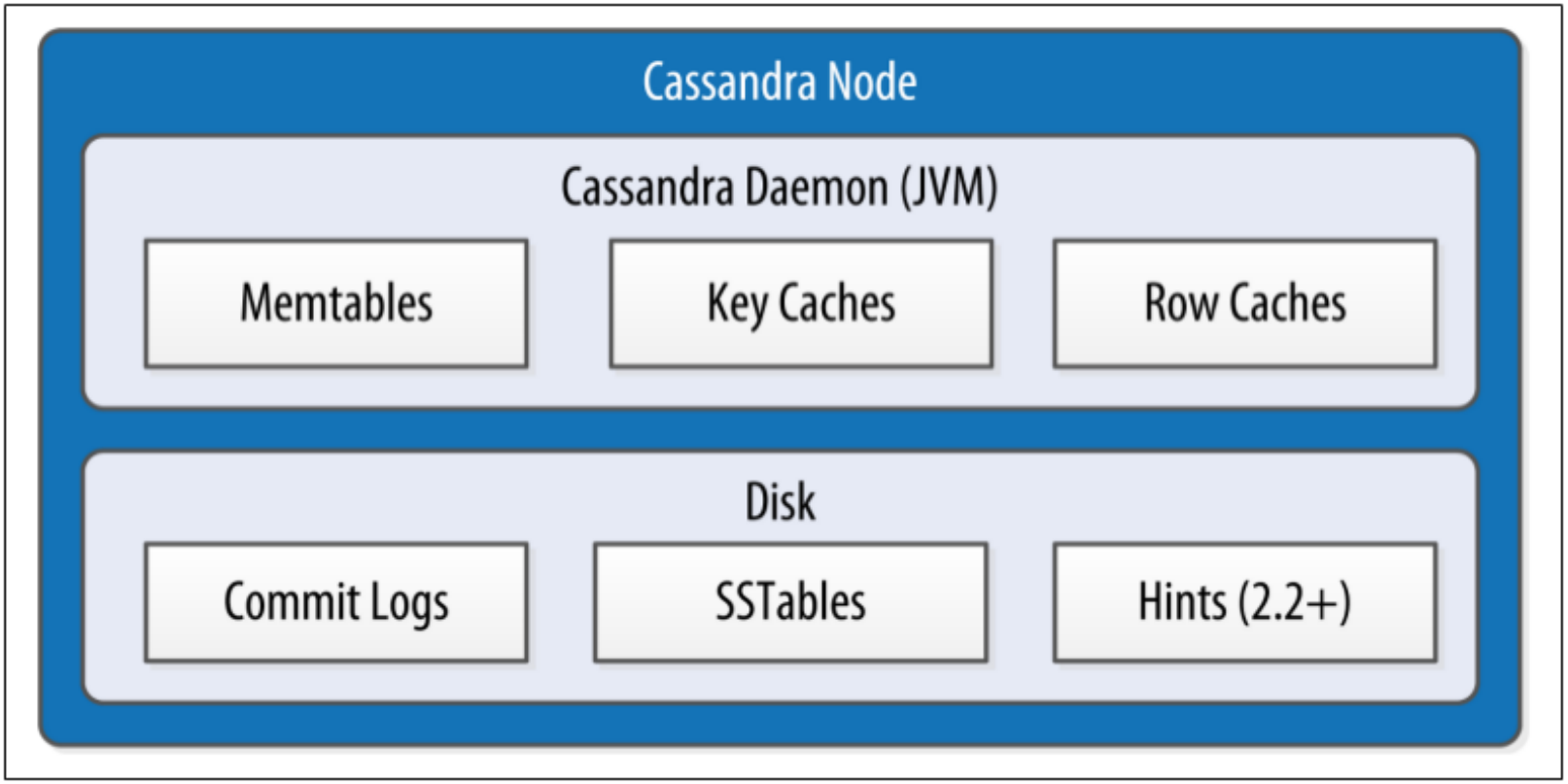


Cassandra – Architecture

- Commit Logs, Memtables, and SSTables
  - Cassandra stores data both in memory and on disk to provide both high performance and durability.
  - Cassandra uses commit logs, memtables, and SSTables to support the writing and reading of data from tables.



Cassandra – Architecture





Cassandra – Architecture

- Commit Logs
  - When a write operation is perform, it is immediately written to a commit log.
  - The commit log is a crash-recovery mechanism that supports Cassandra's durability goals.
  - A write will not count as successful until it's written to the commit log, to ensure that if a write operation does not make it to the in-memory store (the memtable), it will still be possible to recover the data.
  - The database is shut down or it crashes unexpectedly, the commit log can ensure that data is not lost.
  - That's because the next time you start the node, the commit log gets replayed.
  - In fact, that's the only time the commit log is read – clients never read from it.



Cassandra – Architecture

- Commit Logs
  - After it's written to the commit log, the value is written to a memory-resident data structure called the memtable.
  - Each commit log maintains an internal bit flag to indicate whether it needs flushing.
  - When a write operation is first received, it is written to the commit log and its bit flag is set to 1 .
  - There is only one bit flag per table, because only one commit log is ever being written to across the entire server.
  - All writes to all tables will go into the same commit log, so the bit flag indicates whether a particular commit log contains anything that hasn't been flushed for a particular table.



Cassandra – Architecture

- Commit Logs
  - Once the memtable has been properly flushed to disk, the corresponding commit log's bit flag is set to 0, indicating that the commit log no longer has to maintain that data for durability purposes.
  - Like regular logfiles, commit logs have a configurable rollover threshold, and once this file size threshold is reached, the log will roll over, carrying with it any extant dirty bit flags.



Cassandra – Architecture

- Memtables
  - Each memtable contains data for a specific table.
  - In early implementations of Cassandra, memtables were stored on the JVM heap, but now the majority of memtable data was moved to native memory.
  - This makes Cassandra less susceptible to fluctuations in performance due to Java garbage collection.



Cassandra – Architecture

- Memtables
  - When the number of objects stored in the memtable reaches a threshold, the contents of the memtable are flushed to disk in a file called an SSTable.
  - A new memtable is then created.
  - This flushing is a non-blocking operation; multiple memtables may exist for a single table, one current and the rest waiting to be flushed.
  - They typically should not have to wait very long, as the node should flush them very quickly unless it is overloaded.



Cassandra – Architecture

- SSTables
  - Once a memtable is flushed to disk as an SSTable, it is immutable and cannot be changed by the application.
  - Despite the fact that SSTables are compacted, this compaction changes only their on-disk representation.
  - It essentially performs the "merge" step of a mergesort into new files and removes the old files on success.
  - Cassandra supports the compression of SSTables in order to maximize use of the available storage.
  - This compression is configurable per table.
  - Each SSTable also has an associated Bloom filter, which is used as an additional performance enhancer



Cassandra – Architecture

- SSTables
  - All writes are sequential, which is the primary reason that writes perform so well in Cassandra.
  - No reads or seeks of any kind are required for writing a value to Cassandra because all writes are append operations.
  - This makes one key limitation on performance the speed of your disk.
  - Compaction is intended to amortize the reorganization of data, but it uses sequential I/O to do so.



Cassandra – Architecture

- SSTables
  - The write operation is just an immediate append.
  - Compaction helps to organize for better future read performance.
  - If Cassandra naively inserted values where they ultimately belonged, writing clients would pay for seeks up front.
  - On reads, Cassandra will read both SSTables and memtables to find data values, as the memtable may contain values that have not yet been flushed to disk.



Cassandra – Architecture

- Caching
  - Cassandra provides three forms of caching:
  1. Key caches
     - Stores a map of partition keys to row index entries
     - Facilitating faster read access into SSTables stored on disk.
     - The key cache is stored on the JVM heap.
     - The row cache
  2. Row caches
     - Uses entire rows.
     - Can greatly speed up read access for frequently accessed rows, at the cost of more memory usage.
     - The row cache is stored in off-heap memory.
  3. The counter cache
     - Developed to improve counter performance by reducing lock contention for the most frequently accessed counters.



Cassandra – Architecture

- Caching
  - By default, key and counter caching are enabled.
  - Row caching is disabled by default because it requires more memory.
  - Cassandra saves its caches to disk periodically in order to warm them up more quickly on a node restart.



Cassandra – Architecture

- Hinted Handoff
  - A write request is sent to Cassandra, but a replica node where the write properly belongs is not available due to network partition, hardware failure, or some other reason.
  - In this case, the coordinator will create a hint
  - This is used by Cassandra to ensure general availability in such cases.
  - Cassandra holds a separate hint for each partition that is to be written.
  - This allows Cassandra to be always available for writes, and generally enables a cluster to sustain the same write load even when some of the nodes are down.
  - It also reduces the time that a failed node will be inconsistent after it does come back online.